

JavaScript Branching, Arrays & Functional Pipelines

Truthy & Falsy Evaluation

The 6 Falsy Values

Always evaluate to false in boolean contexts.

false Boolean false

0 Numeric zero

"" Empty string

null Intentional absence

undefined Uninitialized value

NaN Not a Number

Truthy Values

Everything not strictly falsy evaluates to true.

[] Empty arrays

{} Empty objects

"false" Non-empty strings

-1 Non-zero numbers

Sequential Branching: if / else

```
if (user.role === 'admin') {  
  grantFullAccess();  
} else if (user.role === 'editor') {  
  grantWriteAccess();  
} else {  
  grantReadAccess();  
}
```

Top-Down Evaluation

Conditions are checked sequentially. First truthy match executes.

Strict Equality

Always prefer `===` over `==` to prevent type coercion.

Fallback Block

The optional `else` block guarantees execution if all fail.

Discrete Matching: The Switch Statement

Exact Value Matches

Compares expression against static cases via strict equality.

Break Necessity

Missing break causes unintended fallthrough to next case.

Default Case

Catch-all execution when no specific cases match.

```
switch (paymentStatus) {  
  case 'PAID':  
    shipOrder();  
    break;  
  case 'PENDING':  
    notifyCustomer();  
    break;  
  default:  
    flagForReview();  
}
```

Inline Expressions: Ternary & Short-Circuit

Ternary Operator (? :)

Replaces simple if/else for direct value assignment.

```
const fee = isMember ? 0 : 20;
```

Logical AND (&&)

Executes right side ONLY if left side is truthy.

```
userLoggedIn && renderDashboard();
```

Logical OR (||)

Returns left side if truthy, otherwise right side fallback.

```
const name = inputName || "Guest";
```



Avoid Nested Ternaries

Chaining ternaries heavily damages code readability.

The Nullish Coalescing Operator (??)

The Problem with OR (||)

Logical OR overrides legitimate falsy values like 0 or "".

```
const quantity = 0; const total = quantity || 10; // total becomes 10  
(incorrect fallback)
```

The Solution: (??)

Only falls back if the left value is strictly null or undefined.

```
const quantity = 0; const total = quantity ?? 10; // total remains 0 (correct  
behavior)
```

Use ?? when 0, "", or false are valid data states.

Memory: Value vs Reference



Primitive Types (Value)

Strings, numbers, booleans are copied by exact value.

```
let a = 10;  
let b = a;  
b = 20;  
  
// a remains 10 (Independent)
```



Array Types (Reference)

Variables hold a pointer to the heap address, not the data.

```
let arr1 = [1, 2];  
let arr2 = arr1;  
arr2.push(3);  
  
// arr1 is now [1, 2, 3] (Shared memory)
```

Stack & Queue Operations

push()

Appends to array tail.

```
arr.push('x'); // adds to end
```

pop()

Removes from array tail.

```
arr.pop(); // removes from end
```

unshift()

Prepends to array head.

```
arr.unshift('a'); // adds to start
```

shift()

Removes from array head.

```
arr.shift(); // removes from start
```

push() / **pop()**: Tail modification.

shift() / **unshift()**: Head modification.

splice(): Inserts or removes targeted ranges.

sort() / **reverse()**: Reorders array in-place.

The Standard for Loop

```
for (let i = 0; i < items.length; i++) {  
  console.log(`Index ${i}:`, items[i]);  
}
```

Initialization (let i = 0)

Executes once. Use let to scope the index variable to the loop block.

Condition (i < length)

Evaluated before each iteration. Loop terminates when falsy.

Final Expression (i++)

Executes after every iteration step to mutate the counter.

Dynamic Termination: while Loops

The while Loop

Checks condition before executing. May run zero times.

```
let i = 0;
while (i < 5) {
  process(i);
  i++; // Must manually update
}
```

The do...while Loop

Executes block first, then checks condition. Guaranteed to run at least once.

```
let i = 0;
do {
  process(i);
  i++;
} while (i < 5);
```

Modern Iterables: for...of vs for...in

for...of (Values)

Use for Arrays & Strings.

Iterates over iterable data structures directly yielding the values.

```
const arr = ['a', 'b'];
for (const val of arr) {
  console.log(val); // 'a', 'b'
}
```

for...in (Keys)

Use for Object Properties.

Iterates over enumerable string properties of an object.

```
const obj = { x: 1, y: 2 };
for (const key in obj) {
  console.log(key); // 'x', 'y'
}
```

Manual Flow Control: Break & Continue

The break Statement

Immediately terminates the entire loop and exits.

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) break;  
  // Stops completely at 5  
}
```

The continue Statement

Skips the remainder of the current iteration step.

```
for (let i = 0; i < 10; i++) {  
  if (i % 2 === 0) continue;  
  // Skips all even numbers  
}
```

First-Class Functions & Callbacks

Functions as Values

In JS, functions are objects. They can be stored in variables.

The Callback Pattern

Passing a function as an argument to another function.

Arrow Function Syntax

Concise syntax ($x \Rightarrow x * 2$) powers functional array methods.

CALLBACK EXAMPLE

```
function processData(data, callback) {  
  // Do some work...  
  const result = data + 10;  
  
  // Execute the passed function  
  callback(result);  
}  
  
// Pass an arrow function as callback  
processData(5, (res) => console.log(res));
```

Side-Effect Execution with forEach()

SYNTAX PATTERN

```
array.forEach((item, index) => {  
  // Execute side effect per item  
  console.log(`${index}: ${item}`);  
});
```

No Return Value

Always returns undefined. Use strictly for side-effects.

Cannot Break Early

Iterates full length. Cannot use break or continue keywords.

Callback Context

Receives element value, current index, and target array reference.

1:1 Element Transformation with map()

INPUT ARRAY

[1, 2, 3, 4]

$x \Rightarrow x * 2$



OUTPUT NEW ARRAY

[2, 4, 6, 8]

Length Invariance

Output array length perfectly matches input array length.

Pure Projection

Maps each element to a new value without mutating source.

```
const prices = [10, 20, 30];
```

```
// Map over prices to add tax  
const withTax = prices.map(p => {  
  return p * 1.2;  
});
```

```
// withTax = [12, 24, 36]
```

Predicate Subset Extraction with filter()

```
const numbers = [ 5, 12, 8, 130, 44 ];  
const filtered = numbers.filter(num => num >= 10);  
// Result: [ 12, 130, 44 ]
```

Boolean Predicate

Callback must return a truthy or falsy evaluation per item.

Variable Output Size

Returns matching subset ranging from 0 to full array length.

Locating Elements: find() & findIndex()

find()

Returns the first element that satisfies the predicate. Returns undefined if none match.

```
const users = [{id: 1}, {id: 2}];  
const user = users.find(u => u.id === 2);  
// Returns: {id: 2}
```

findIndex()

Returns the index of the first matching element. Returns -1 if none match.

```
const scores = [10, 20, 30];  
const idx = scores.findIndex(s => s > 15);  
// Returns: 1
```

Predicate Testing: `some()` & `every()`

✓ `some()`

Returns true if AT LEAST ONE element satisfies the predicate.

```
const nums = [1, 2, 3, 4];  
const hasEven = nums.some(n => n % 2 === 0);  
// true
```

✓✓ `every()`

Returns true ONLY if ALL elements satisfy the predicate.

```
const nums = [2, 4, 6];  
const allEven = nums.every(n => n % 2 === 0);  
// true
```

State Aggregation with reduce()

REDUCER PATTERN

```
const total = [ 10, 20, 30 ].reduce(  
  (accumulator, current) => {  
    return accumulator + current;  
  },  
  0 // Initial Value  
);  
// Result: 60
```

Accumulator State

Passes updated running value across successive iterations.

Initial Value Safety

Always supply seed accumulator to prevent empty array errors.

Sorting Arrays: The String Gotcha

Default String Sort

By default, sort converts to strings. Numbers sort incorrectly.

```
const nums = [10, 2, 30];  
nums.sort();  
// [10, 2, 30] (Alphabetical!)
```

Numeric Comparator

Provide a comparison function for accurate numeric sorting.

```
const nums = [10, 2, 30];  
nums.sort((a, b) => a - b);  
// [2, 10, 30] (Numeric!)
```

Method Chaining & Data Composition

STEP 1: FILTER

Extract active items



STEP 2: MAP

Extract price values



STEP 3: REDUCE

Compute total sum

```
const totalActiveValue = products
  .filter(p => p.inStock)
  .map(p => p.price)
  .reduce((sum, price) => sum + price, 0);
```